Informe técnico · AsesorIA *Integración de TDD y BDD en un pipeline CI/CD con SonarCloud y JaCoCo*

Estudiante	Fecha	Proyecto / funcionalidad	Stack
Manuela Larrea	26 de septiembre de 2026	AsesorIA · Guardrail de datos sensibles como servicio (RF-05, RF-06)	Java 17 · JUnit 5 · JaCoCo 0.8 · Cucumber 7 · Playwright 1.56 · Python 3.11 · FastAPI · pytest · GitHub Actions · SonarCloud

1. Contexto y funcionalidad seleccionada

AsesorIA es el asistente conversacional con IA para asesores financieros construido a lo largo del curso (Unidad 1: plan y suite JUnit 5 + Playwright; Unidad 2: escenarios BDD en Gherkin). Para esta unidad se seleccionó la funcionalidad de mayor riesgo regulatorio del producto: el guardrail de datos sensibles, que impide que números de tarjeta (PAN) o cuentas (IBAN) lleguen al motor de IA y garantiza que el historial solo conserve versiones enmascaradas (requisitos RF-05 y RF-06). En una entidad financiera este control es transversal a todos los canales que consumen modelos de lenguaje, por lo que la decisión de diseño fue extraerlo del monolito Python a un servicio independiente en Java (guardrail-service), desarrollado íntegramente con TDD, con cobertura medida por JaCoCo y calidad evaluada por SonarCloud, y verificado de extremo a extremo por los escenarios BDD ya existentes.

Componente	Tecnología	Rol en el flujo de pruebas
guardrail-service (nuevo)	Java 17, JDK HttpServer, sin dependencias de runtime; Maven; JaCoCo	Unidad bajo TDD: Luhn, detector PAN/IBAN, política, JSON y contrato HTTP (30 pruebas JUnit 5).
SUT AsesorIA	Python 3.11, FastAPI, scikit-learn	Consume el servicio vía GUARDRAIL_URL con degradación al control local; 31 pruebas pytest (unitarias + componente con TestClient).
Especificación BDD	Gherkin (es), Cucumber-JVM 7 + Playwright; verificación cruzada con Cucumber.js	HU-01 con 6 escenarios / 9 ejecuciones que validan el mismo flujo desde la interfaz.
Suite de regresión	JUnit 5 + Playwright (Unidad 1)	32 pruebas API y UI que garantizan que la extracción no rompió comportamiento previo.
Pipeline	GitHub Actions, SonarCloud, JaCoCo, coverage.py	Cinco jobs: TDD Java, TDD Python, análisis Sonar con quality gate, BDD/E2E con servicios reales, resumen.

2. Desarrollo guiado por pruebas (TDD)

El servicio se construyó en cuatro ciclos rojo-verde-refactor, cada uno registrado en Git como un par de commits (test → feat) para que la historia del repositorio sea la evidencia del proceso: la prueba siempre precede al código. En la fase roja las clases existen como stubs que lanzan UnsupportedOperationException, de modo que la suite compila y falla por el motivo correcto.

Ciclo	Prueba escrita primero (RED)	Implementación mínima (GREEN)	Refactor / decisión
1	LuhnTest: 4 PAN válidos, 3 inválidos, entradas vacías/no numéricas (8 pruebas)	Luhn.isValid con recorrido inverso y duplicación alterna	Se mantiene sin estado (clase final, utilitaria).
2	SensitiveDataDetectorTest: PAN con espacios/guiones, IBAN CR, múltiples hallazgos, texto limpio, falso positivo sin Luhn (6)	Expresiones regulares + confirmación Luhn; Finding como record	Enmascaramiento delegado al detector para reutilizarlo desde la política.
3	GuardrailPolicyTest: permitido, bloqueado, vacío, nulo, valores límite 500/501 (6)	GuardrailPolicy.evaluate → Decision(ALLOWED/BLOCKED/INVALID)	Mensajes de negocio idénticos a los del SUT Python para no cambiar el contrato observable.
4	JsonTest + GuardrailServerTest: contrato HTTP (200/400/405, /health) y (de)serialización (10)	JSON mínimo propio y servidor JDK sin dependencias	El reporte JaCoCo mostró ramas de unescape sin cubrir → 2 pruebas adicionales (93,5 % líneas).

git log — ciclos TDD RED → GREEN → REFACTOR (guardrail-service)

```
$ git log --reverse --oneline --date=short
e12787e 2026-09-26 Unidades 1 y 2: SUT AsesorIA, harness JUnit 5 + Playwright y escenarios BDD (Cucumber)
6f6b9a6 2026-09-26 test(guardrail): RED - LuhnTest define el contrato de validación de PAN (8 pruebas fallan)
e382d1b 2026-09-26 feat(guardrail): GREEN - implementación mínima del algoritmo de Luhn
d6ede6c 2026-09-26 test(guardrail): RED - SensitiveDataDetectorTest (PAN con Luhn, IBAN, enmascarado, falsos positivos)
f6bcf7b 2026-09-26 feat(guardrail): GREEN - detector de PAN/IBAN con enmascaramiento
83daf3f 2026-09-26 test(guardrail): RED - GuardrailPolicyTest (permitir, bloquear, inválido, valores límite)
ba4d51c 2026-09-26 feat(guardrail): GREEN - política de evaluación (ALLOWED / BLOCKED / INVALID)
21f9e3b 2026-09-26 test(guardrail): RED - JsonTest y GuardrailServerTest definen el contrato HTTP/JSON
fcd76ed 2026-09-26 feat(guardrail): GREEN - JSON mínimo y servidor HTTP (JDK HttpServer) · 28 pruebas · JaCoCo 87 % líneas
6c511da 2026-09-26 test(guardrail): REFACTOR - casos de borde de Json guiados por el reporte JaCoCo (93,5 % líneas, 88,4 % ramas)
707276a 2026-09-26 feat(sut): integra guardrail-service (GUARDRAIL_URL con degradación local), pruebas de componente con TestClient (99
8a45368 2026-09-26 ci: pipeline GitHub Actions (TDD Java+JaCoCo, TDD Python, SonarCloud quality gate, BDD/E2E) y sonar-project.properti
0985ce8 2026-09-26 chore(sut): correcciones de análisis estático (ruff I001/RUF100, pylint W1508) y falso positivo documentado (bandit
```

Figura 1. Historial Git: cada ciclo TDD como par de commits test (RED) → feat (GREEN), más el refactor guiado por JaCoCo.

TDD ciclo 1 · fase RED — LuhnTest contra el stub (8/8 fallan)

```
$ java -jar junit-platform-console-standalone.jar execute --scan-classpath  (guardrail-service)
Thanks for using JUnit! Support its development at https://junit.org/sponsoring

╷
├─ JUnit Platform Suite ✔
├─ JUnit Jupiter ✔
│  └─ Luhn ✔
│     ├─ rechazaNumerosInvalidos(String) ✔
│     │  ├─ rechaza 4111111111111112 ✘ pendiente (TDD: fase roja)
│     │  ├─ rechaza 1234567890123 ✘ pendiente (TDD: fase roja)
│     │  └─ rechaza 0000000000000001 ✘ pendiente (TDD: fase roja)
│     ├─ aceptaNumerosValidos(String) ✔
│     │  ├─ acepta PAN válido 4111111111111111 ✘ pendiente (TDD: fase roja)
│     │  ├─ acepta PAN válido 5500000000000004 ✘ pendiente (TDD: fase roja)
│     │  ├─ acepta PAN válido 340000000000009 ✘ pendiente (TDD: fase roja)
│     │  └─ acepta PAN válido 6011000000000004 ✘ pendiente (TDD: fase roja)
│     └─ rechaza entradas vacías o con caracteres no numéricos ✘ pendiente (TDD: fase roja)
└─ JUnit Vintage ✔
...
  => java.lang.UnsupportedOperationException: pendiente (TDD: fase roja)
     edu.testing.asesoria.guardrail.Luhn.isValid(Luhn.java:9)

Test run finished after 240 ms
[         8 tests found           ]
[         0 tests skipped         ]
[         8 tests started         ]
[         0 tests aborted         ]
[         0 tests successful      ]
[         8 tests failed          ]
```

Figura 2. Ciclo 1 en fase roja: las 8 pruebas de LuhnTest fallan contra el stub por el motivo esperado.

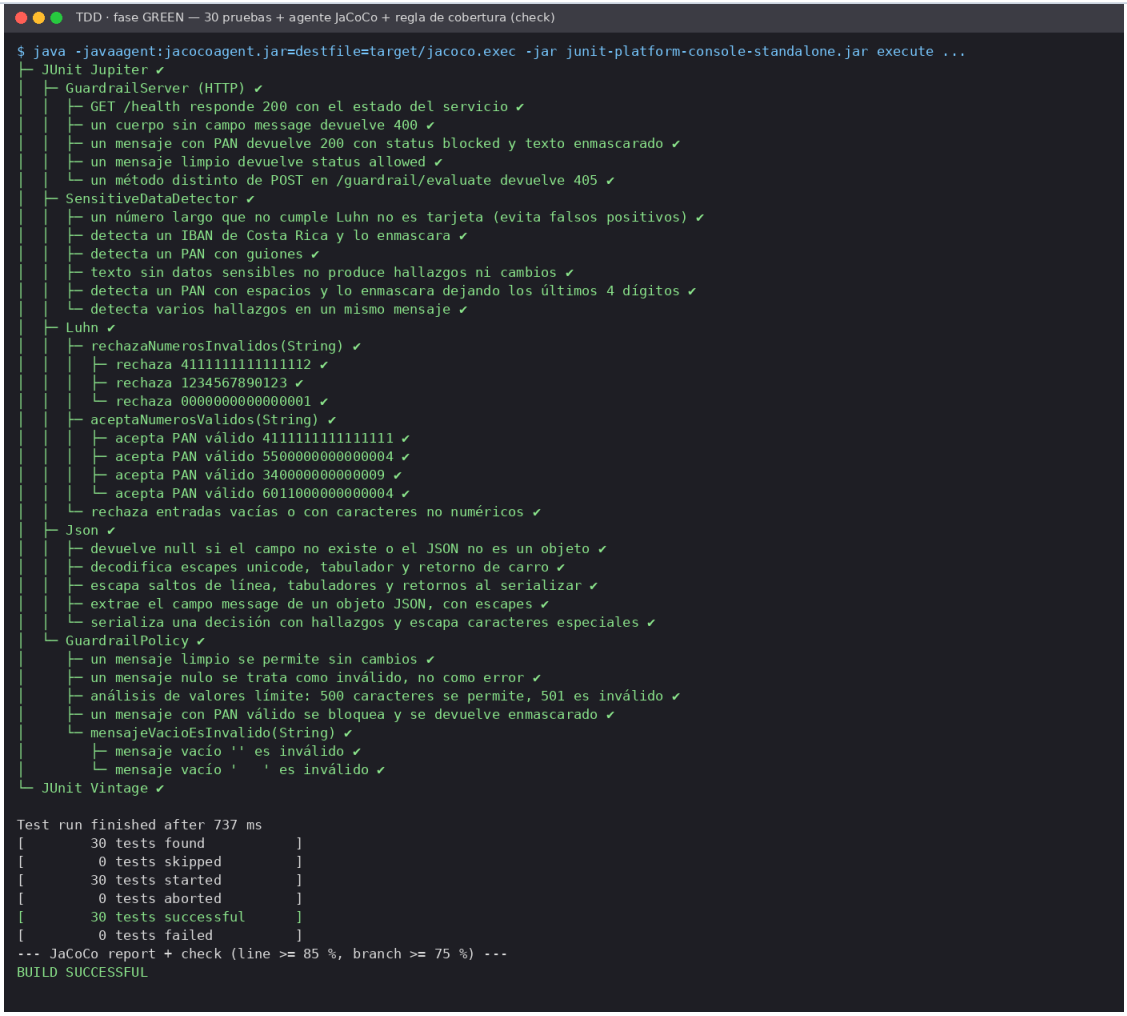


Figura 3. Suite completa en verde (30 pruebas) ejecutada con el agente JaCoCo y regla de cobertura mínima (BUILD SUCCESSFUL).

3. Especificación y automatización del comportamiento (BDD)

La especificación de la Unidad 2 (HU-01, criterios CA-01 a CA-03) se amplió con un esquema de escenario que recorre los tipos de dato sensible soportados por el nuevo servicio (PAN con guiones e IBAN nacional). Los escenarios no cambiaron de vocabulario: siguen hablando de lo que hace el asesor y lo que responde el asistente, no de servicios ni puertos; por eso el mismo .feature valida hoy una arquitectura distinta a la de hace dos semanas. El glue Java (Cucumber-JVM) reutiliza los Page Objects; como verificación cruzada de que la especificación es independiente de la implementación, el mismo archivo se ejecutó también con un glue JavaScript (Cucumber.js + Playwright) contra el servicio y el SUT reales.

Criterio	Escenario Gherkin	Pruebas TDD que cubren la misma regla	Resultado
CA-01 respuesta oficial con intención	«Una consulta del dominio…» + esquema de 4 paráfrasis	pytest test_nlu (clasificación, 6 casos)	5/5 ✓
CA-02 fallback fuera de dominio	«…recibe una respuesta de derivación, no una inventada»	pytest test_nlu (fallback, 4 casos)	1/1 ✓
CA-03 datos sensibles bloqueados y enmascarados	«…con número de tarjeta se bloquea…» + esquema PAN/IBAN (2 ejemplos)	JUnit LuhnTest, SensitiveDataDetectorTest, GuardrailPolicyTest, GuardrailServerTest (30) · pytest test_guardrail_integration (6)	3/3 ✓



Figura 4. Reporte Cucumber del feature consulta_asistente.feature (Gherkin en español): 9 escenarios, 50 pasos, 100 % en verde, con captura adjunta por escenario.



Figura 5. Ejecución en consola de los escenarios BDD contra el servicio Java y el SUT reales (4,2 s).

4. Trazabilidad entre requisitos, código y comportamiento

La matriz enlaza cada requisito con la clase que lo implementa, la prueba unitaria que lo fijó primero, el escenario Gherkin que lo valida desde la interfaz y el commit que lo introdujo. Los identificadores viajan en el código (@DisplayName, etiquetas @CA-xx, mensajes de commit), de modo que cualquier reporte del pipeline puede rastrearse hasta el requisito.

Requisito	Código (guardrail-service / SUT)	Prueba unitaria (TDD)	Escenario BDD	Commit
RF-05 bloqueo y enmascarado de PAN	Luhn, SensitiveDataDetector.detect/mask, GuardrailPolicy	LuhnTest, SensitiveDataDetectorTest, GuardrailPolicyTest	@CA-03 escenario + esquema (fila PAN)	6f6b9a6 … ba4d51c
RF-05 bloqueo y enmascarado de IBAN	SensitiveDataDetector (patrón IBAN CR)	detectaIbanNacional, detectaVariosHallazgos	@CA-03 esquema (fila IBAN)	d6ede6c, f6bcf7b
RF-06 validación de entrada (vacío, ≤ 500)	GuardrailPolicy.evaluate; guardrails.evaluate (Python)	limiteDeLongitud, mensajeVacioEsInvalido; test_evaluacion_local_valida_longitud	CP-API-03/04 (suite Unidad 1)	83daf3f, ba4d51c
RNF-02 PII nunca en claro	Decision.maskedMessage; main.py almacena solo el texto enmascarado	evaluaMensajeConPan (HTTP); test_chat_flujo_completo	@CA-03 «el historial muestra… enmascarado»	fcd76ed, 707276a
RNF-04 resiliencia del control	guardrails.evaluate: degradación a local si el servicio falla	test_si_el_servicio_falla_degrada_a_local	— (cubierto en unidad; candidato a escenario de caos)	707276a

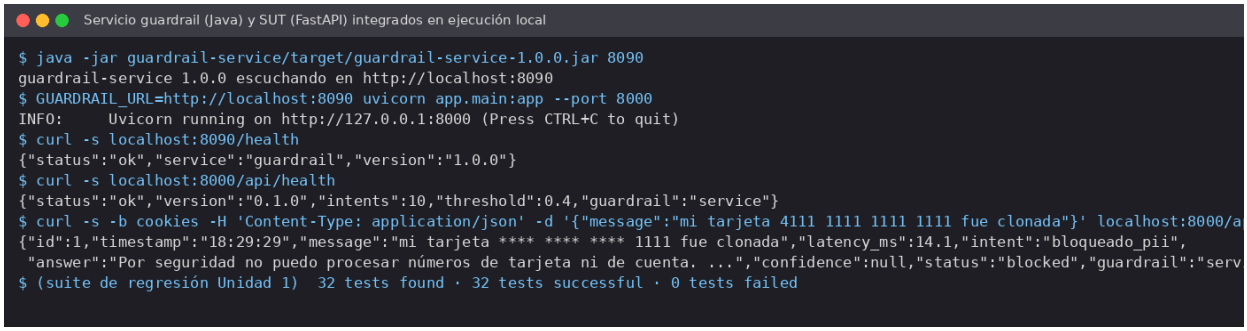


Figura 6. Integración real: el SUT informa guardrail=service, la consulta con PAN vuelve bloqueada y enmascarada, y la suite de regresión de la Unidad 1 sigue en verde (32/32).

5. Pipeline CI/CD

El flujo se implementó en GitHub Actions (.github/workflows/ci.yml) con cinco jobs y dependencias explícitas: los dos jobs de TDD corren en paralelo y publican sus reportes como artefactos; el análisis de SonarCloud los consume y espera el resultado del quality gate (el pipeline falla si no se cumple); el job BDD levanta el jar del servicio y el SUT reales, instala Chromium y ejecuta Cucumber-JVM junto con la suite de regresión; un último job escribe el resumen. La configuración de Sonar vive en sonar-project.properties (monorepo Java + Python con rutas a jacoco.xml y coverage.xml).

Job	Herramientas	Salida / gate
tdd-java	setup-java 17 · mvn verify · jacoco:report + jacoco:check	Falla si líneas < 85 % o ramas < 75 %; artefactos: reporte JaCoCo, surefire, jar del servicio.
tdd-python	setup-python 3.11 · pytest --cov --cov-fail-under=85	Falla si cobertura < 85 %; artefactos: coverage.xml, pytest-report.xml.
sonar	sonarqube-scan-action + sonarqube-quality-gate-action	Quality gate «Sonar way» sobre código nuevo; requiere SONAR_TOKEN.
bdd-e2e	java -jar guardrail-service · uvicorn con GUARDRAIL_URL · Playwright install · mvn test	Cucumber (html/json), Surefire XML publicado como check, capturas y videos como artefactos.
resumen	GITHUB_STEP_SUMMARY	Tabla con el resultado de cada job.

```
sonar:
  needs: [tdd-java, tdd-python]
  steps:
    - uses: actions/download-artifact@v4   # jacoco.xml + coverage.xml de los jobs TDD
    - uses: SonarSource/sonarqube-scan-action@v5
      env: { SONAR_TOKEN: ${{ secrets.SONAR_TOKEN }} }
    - uses: SonarSource/sonarqube-quality-gate-action@v1   # bloquea el pipeline si el gate falla
```

[PENDIENTE] Captura de GitHub Actions – ejecución del workflow ci

Reemplazar por la captura de la pestaña Actions del repositorio tras el primer push:
· vista del run con los 5 jobs en verde (tdd-java, tdd-python, sonar, bdd-e2e, resumen)
· pestaña Summary con las tablas de cobertura y duración por job
Guía paso a paso en el Anexo B del informe.

Figura 7. Ejecución del workflow en GitHub Actions (pendiente de captura tras el primer push; ver Anexo B).

6. Métricas de calidad

Cobertura y gates.

Módulo	Herramienta	Pruebas	Cobertura líneas	Cobertura ramas	Gate
guardrail-service (Java)	JaCoCo 0.8.11	30 JUnit 5	116/124 · 93,5 %	61/69 · 88,4 %	≥ 85 % / ≥ 75 % ✓
SUT AsesorIA (Python)	coverage.py 7 vía pytest-cov	31 pytest	242/243 · 99 % (antes de las pruebas de componente: 51 %)	n/a	≥ 85 % ✓
Escenarios BDD	Cucumber	9 escenarios / 50 pasos	3/3 criterios de aceptación	—	100 % ✓
Regresión API + UI	JUnit 5 + Playwright	32	11/11 requisitos	—	100 % ✓

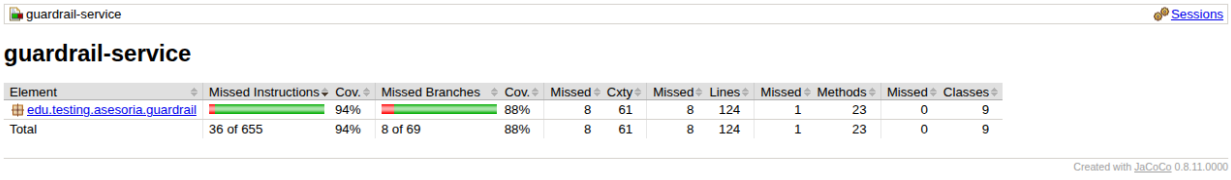


Figura 8. Reporte JaCoCo de guardrail-service: 94 % de instrucciones, 88 % de ramas, 124 líneas, 9 clases.



Figura 9. Detalle JaCoCo de Json.java: las líneas rojas/amarillas guiaron el refactor del ciclo 4 (casos de escape no cubiertos).

pytest · SUT AsesorIA — 31 pruebas con cobertura (coverage.py)

```
$ pytest -q tests --cov=app --cov-report=term --cov-report=xml --junitxml=pytest-report.xml
.............................                                            [100%]
============================== warnings summary ==============================
.venv/lib/python3.11/site-packages/fastapi/testclient.py:1
  /home/claude/asesoria-testing/sut/.venv/lib/python3.11/site-packages/fastapi/testclient.py:1: StarletteDeprecationWarning: Using `htt
    from starlette.testclient import TestClient as TestClient  # noqa

-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
================================ tests coverage ================================
_______________ coverage: platform linux, python 3.11.15-final-0 _______________

Name                Stmts   Miss  Cover
-------------------------------------------
app/__init__.py         1      0   100%
app/auth.py            27      0   100%
app/guardrails.py      73      0   100%
app/main.py            89      1    99%
app/nlu.py             53      0   100%
-------------------------------------------
TOTAL                 243      1    99%
Coverage HTML written to dir htmlcov
Coverage XML written to file coverage.xml
31 passed, 1 warning in 2.39s
```

Figura 10. pytest con cobertura del SUT: 31 pruebas, 99 % de líneas (51 % antes de añadir las pruebas de componente).

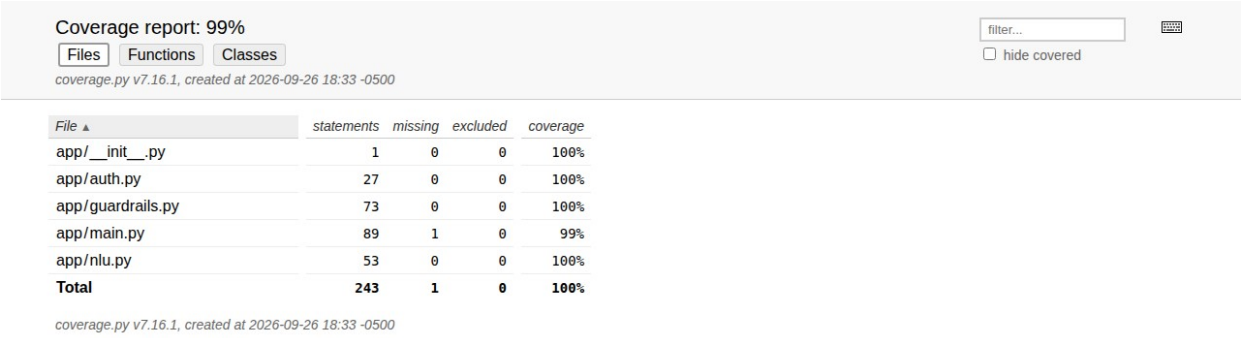


Figura 11. Reporte HTML de coverage.py por módulo.

Deuda técnica, code smells y falsos positivos.

Mientras SonarCloud no se ejecute en la nube (Anexo B), se corrió localmente un conjunto de analizadores equivalentes para anticipar los hallazgos y decidir cuáles corregir, aceptar o marcar como falsos positivos. La deuda técnica estimada por Sonar para el código nuevo se espera baja: el servicio tiene 124 líneas, complejidad ciclomática total 61 y sin duplicación; la principal deuda aceptada es la documentación Javadoc de métodos públicos (≈ 5 × 5 min según el modelo SQALE de Sonar ≈ 25 min).

Herramienta (local)	Hallazgos iniciales	Acción	Estado final
ruff (Python)	3: I001 imports desordenados, RUF100 noqa sin uso, DTZ005 datetime.now sin zona	2 corregidos con --fix; DTZ005 aceptado: la hora se muestra solo en la sesión del asesor (hora local)	1 aceptado
pylint (Python)	W1508 valor por defecto no string en os.getenv → 9,96/10	Corregido (str(DEFAULT_THRESHOLD))	10,00/10
bandit (seguridad Python)	B310 urllib.urlopen (Medium)	Falso positivo: la URL proviene de configuración interna y el esquema es fijo → # nosec B310 con justificación	0 abiertos
checkstyle google_checks (Java)	25 avisos: 13 LineLength (100 col.), 5 MissingJavadocMethod, 2 SingleLineJavadoc, 2 RightCurlyAlone, 3 otros	LineLength: falso positivo por convención (el proyecto usa 120 col.); Javadoc en métodos públicos: deuda menor aceptada para esta iteración	12 aceptados · 13 falsos positivos

Análisis estático local (proxy de code smells / hotspots previo a SonarCloud)

```
$ ruff check app tests           # antes: 3 hallazgos (I001, RUF100, DTZ005) → después: 1 (DTZ005, aceptado)
Found 1 error.
$ pylint app --max-line-length=120  # antes 9.96/10 (W1508) → después:
Your code has been rated at 10.00/10
$ bandit -q -r app               # B310 urlopen: falso positivo documentado con # nosec
Total issues (by severity): Medium: 0  High: 0   (1 skipped by #nosec B310)
$ checkstyle -c google_checks.xml guardrail-service/src/main/java
25 warnings: 13 LineLength(100) · 5 MissingJavadocMethod · 2 SingleLineJavadoc · 2 RightCurlyAlone · 1 SummaryJavadoc · 1 RequireEmptyL
```

Figura 12. Análisis estático local antes/después: ruff, pylint, bandit y checkstyle.

[PENDIENTE] Captura de SonarCloud – Overview y Quality Gate

Reemplazar por la captura del proyecto en sonarcloud.io tras el análisis:

- Overview: Quality Gate (Passed), Bugs, Vulnerabilities, Security Hotspots, Code Smells, Coverage, Duplications y Technical Debt
- Issues: lista con las alertas y las marcadas como False Positive / Accepted

Figura 13. SonarCloud: quality gate, deuda técnica y code smells (pendiente de captura tras el análisis en la nube; ver Anexo B).

Tiempos de ejecución.

Etapa	Duración medida (local)	Observación
TDD Java (30 pruebas + JaCoCo + check)	0,74 s pruebas · ≈ 2 s con reporte	Sin dependencias externas: el ciclo rojo-verde tarda segundos.
TDD Python (31 pruebas + cobertura)	2,4 s	Incluye arranque de la app con TestClient.
BDD Cucumber (9 escenarios, Chromium headless)	4,2 s	Servicio Java + SUT reales.
Regresión JUnit + Playwright (32 pruebas)	8,7 s	Un contexto de navegador por prueba.
Pipeline completo en GitHub Actions	≈ 6–8 min (estimado)	Dominado por la descarga de dependencias Maven/pip y de Chromium; valor real a registrar tras la primera ejecución (Anexo B).

7. Análisis reflexivo: TDD + BDD, trazabilidad, colaboración y calidad

Trazabilidad. La cadena requisito → prueba unitaria → código → escenario → commit de la sección 4 no se construyó al final: es el subproducto natural de haber escrito primero la prueba (TDD) y de haber mantenido la especificación en Gherkin (BDD). Cuando el guardrail cambió de un módulo Python a un servicio Java, ninguna frase del .feature tuvo que reescribirse y los identificadores CA-xx siguieron enlazando la historia de usuario con el reporte de Cucumber; TDD aportó el otro extremo de la cadena, el commit que introdujo cada regla. Para un entorno regulado esto significa que la evidencia de cumplimiento (por ejemplo, «los PAN nunca se almacenan en claro») se produce en cada ejecución del pipeline en lugar de recopilarse manualmente antes de una auditoría.

Colaboración. Los dos enfoques hablan a audiencias distintas y se complementan: los escenarios BDD son el contrato con negocio y con el oficial de cumplimiento (qué se bloquea y qué se muestra), mientras que las pruebas TDD son el contrato entre desarrolladores (qué hace Luhn con un guion, qué devuelve el servicio ante un cuerpo inválido). La verificación cruzada con dos glues demostró que la especificación es el activo estable y la implementación es intercambiable, un argumento útil cuando equipos con stacks diferentes (Python para IA, Java para servicios corporativos) deben converger sobre el mismo comportamiento.

Calidad. Las métricas mostraron dónde estaba la deuda real: la cobertura del SUT pasó de 51 % a 99 % no por perseguir un número, sino porque las pruebas de componente con TestClient cubrían rutas web que ninguna prueba unitaria tocaba; el reporte JaCoCo señaló ramas de escape JSON sin ejercitar y motivó dos pruebas más. Los quality gates automatizados (JaCoCo check, cov-fail-under, Sonar) convierten esos umbrales en criterios objetivos de mezcla, y el registro de falsos positivos (bandit B310, LineLength) evita que el equipo silencie reglas sin justificación escrita. La limitación principal es que ni TDD ni BDD miden la calidad del modelo de IA: la precisión del clasificador sigue requiriendo datasets y métricas propias, que en una siguiente iteración deberían publicarse también como artefactos del pipeline.

Anexo A. Reproducción local

- **Servicio y SUT:** cd guardrail-service && mvn verify && java -jar target/guardrail-service-1.0.0.jar 8090 · cd sut && GUARDRAIL_URL=http://localhost:8090 uvicorn app.main:app --port 8000.
- **TDD:** mvn verify (JUnit 5 + JaCoCo report + check) · pytest -q tests --cov=app --cov-fail-under=85.

- **BDD y regresión:** cd tests-e2e && mvn test (o -Dtest=RunCucumberTest); verificación cruzada: cd tests-bdd-js && npm install && npm test.
- **Pipeline:** push a main o workflow_dispatch en GitHub Actions; requiere el secreto SONAR_TOKEN y los valores de organización/proyecto en sonar-project.properties.

Anexo B. Pasos para completar la evidencia en la nube

- Crear el repositorio en GitHub (público o privado) y hacer push de la rama main con el historial incluido en el zip (.git).
- En sonarcloud.io: importar el repositorio, elegir «Analysis method: GitHub Actions», copiar el token en Settings → Secrets → SONAR_TOKEN del repositorio y actualizar sonar.organization / sonar.projectKey en sonar-project.properties.
- Ejecutar el workflow (Actions → ci → Run workflow). Capturar: (a) el run con los cinco jobs, (b) la pestaña Summary, (c) el artefacto evidencia-bdd-e2e.
- En SonarCloud capturar Overview (quality gate, coverage, code smells, technical debt) e Issues; marcar como False Positive las alertas justificadas en la sección 6 y volver a capturar.
- Sustituir las figuras 12 y 13 de este informe por las capturas anteriores y completar la fila «Pipeline completo» de la tabla de tiempos.